

Нейросетевые представления видео

от 2D CNN до видео-трансформеров и self-supervised learning

Александр Савельев

Курс: Обработка видео в контексте машинного обучения и ИИ
Лекция 3

План лекции

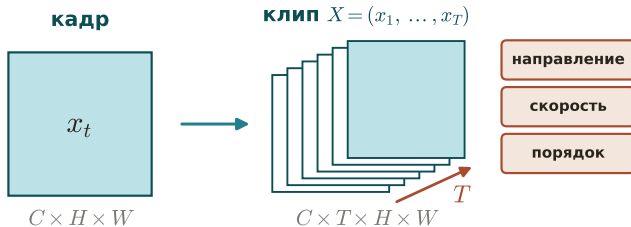
Главный вопрос: *где и как модель смешивает информацию между кадрами — и сколько это стоит.*

1. Временная ось: чего нет в одном кадре
2. 2D CNN по кадрам: average / max / LSTM; Two-stream (RGB + optical flow)
3. 3D CNN: C3D, I3D, R(2+1)D; эффективные: SlowFast, X3D, TSN
4. Видео-трансформеры: токенизация, TimeSformer, ViViT, Video Swin, MViT
5. Self-supervised: VideoMAE, контрастное обучение, InternVideo2, V-JEPA
6. Датасеты и appearance bias, сэмплирование кадров, test-time views, метрики
7. Как выбрать модель на практике

1. Видео — тензор с осью времени

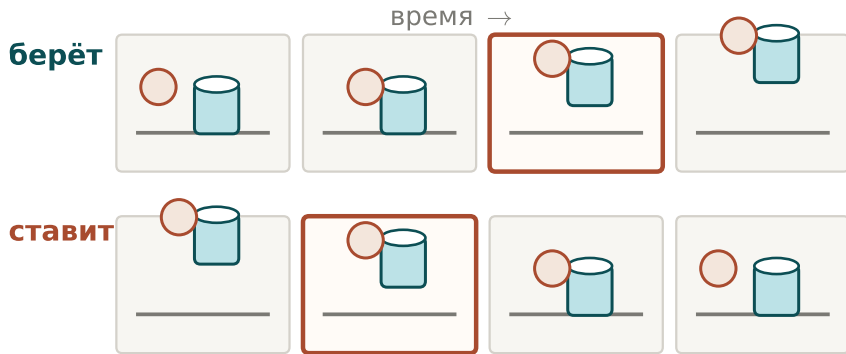
Обозначения

Изображение: $x \in \mathbb{R}^{C \times H \times W}$. Видео: $X = (x_1, \dots, x_T)$, $X \in \mathbb{R}^{C \times T \times H \times W}$.



Интуиция. Ось T несёт направление, скорость, порядок событий и взаимодействие объектов. Но соседние кадры почти одинаковы: время нужно использовать, не сделав модель в разы дороже.

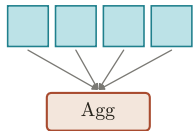
1. Один кадр не отличает «берёт» от «ставит»



Обведённые кадры почти совпадают. Класс действия определяется *порядком* кадров, а не содержанием одного из них.

1. Три семейства моделей

2D CNN + агрегация



каждый кадр отдельно,
время только в Agg

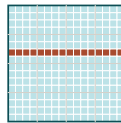
3D-свёртка



локальное окно
по t, x, y

трансформер

матрица $N \times N$



любой токен с любым,
 $N = T \cdot H \cdot W$

Почему трансформеру нужна экономия

Токенов $N = T \cdot H \cdot W$ (в единицах патчей), полное self-attention стоит $O(N^2) = O((THW)^2)$.

2D CNN + агрегация — дёшево, но движение почти не видно. 3D CNN — локальное движение, дороже в k_t раз. Трансформер — глобальные связи, квадратичная цена.

2. Бейзлайн: 2D CNN по кадрам

Схема (например, ResNet-50 с весами ImageNet)

$$z_t = f_{\theta}(x_t) \in \mathbb{R}^d, \quad z = \text{Agg}(z_1, \dots, z_T), \quad \hat{y} = Wz + b.$$

Плюсы

- ▶ backbone из ImageNet — без изменений
- ▶ цена $\approx T$ прогонов image-модели
- ▶ число параметров не зависит от T

Минусы

- ▶ свёртки видят один кадр
- ▶ фильтры не учатся реагировать на сдвиг пикселей
- ▶ всё время — только в Agg

Интуиция. Сначала каждый кадр описывают вектором, потом векторы как-то склеивают. Вопрос лекции — как именно склеивать и можно ли раньше.

2. Агрегаторы: average, max, LSTM

Average

$$z = \frac{1}{T} \sum_t z_t$$

«мешок кадров»

Max

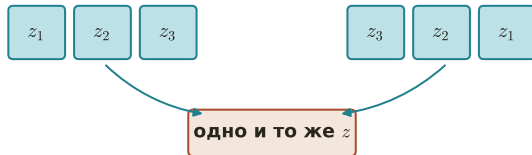
$$z_j = \max_t z_{t,j}$$

редкие события; боится выбросов

LSTM / GRU

$$h_t = \text{LSTM}(z_t, h_{t-1})$$

класс по h_T ; порядок важен



Average хватает, когда класс виден по сцене («игра на гитаре»), но «положить» и «снять» после усреднения совпадают. LSTM видит порядок *ГОТОВЫХ* векторов; CNN под ним движение пикселей всё ещё не видит.

2. Two-stream Simonyan, Zisserman, 2014

Два потока

Spatial: RGB-кадр $\rightarrow$ внешний вид.

Temporal: оптический поток

$$F_t(u, v) = (\Delta u, \Delta v).$$

L полей потока $\Rightarrow$ вход из $2L$ каналов.

Fusion

$$\text{Late: } s = \lambda s^{\text{rgb}} + (1 - \lambda) s^{\text{flow}}$$

$$\text{Early / mid: } z = [z^{\text{rgb}}; z^{\text{flow}}]$$

Плюсы

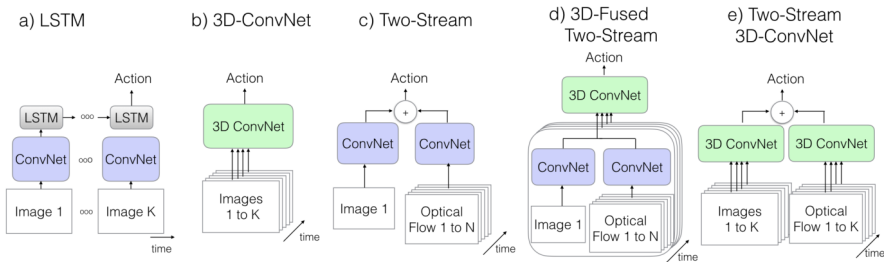
- ▶ движение подаётся явно
- ▶ меньше зависимость от внешнего вида
- ▶ flow давал +несколько п.п. к RGB

Минусы

- ▶ классический поток (KLT и др.) бывает медленнее самой сети
- ▶ две ветви на инференсе
- ▶ не end-to-end

Интуиция. Разделить «что в кадре» и «что движется». Flow потом вытеснили 3D CNN и SSL, а идея двух ветвей вернулась в SlowFast.

2. Ранние архитектуры на одной схеме



Carreira, Zisserman, CVPR 2017, Fig. 2

(a) 2D CNN + LSTM, (b) 3D ConvNet (C3D), (c) Two-stream с late fusion, (d) 3D-fused two-stream, (e) Two-stream I3D — лучшая в статье.

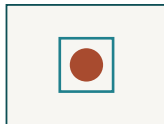
3. 3D-свёртка: фильтр видит соседние кадры

Ядро и выход

2D: $K \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$, 3D: $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_t \times k \times k}$

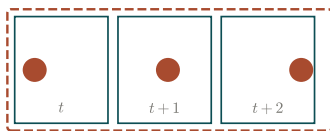
$$Y_{c_o, t, h, w} = \sum_{c_i, \tau, i, j} K_{c_o, c_i, \tau, i, j} X_{c_i, t+\tau, h+i, w+j}$$

2D: один кадр



сдвиг не виден

3D: $k_t = 3$ кадра сразу



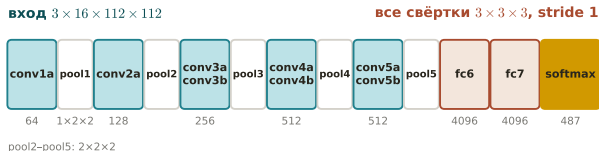
одно ядро покрывает все три кадра

Интуиция. Движение — это паттерн в объёме $t \times h \times w$. Фильтр размера $k_t \times k \times k$ реагирует на движущуюся границу, жест, появление объекта.

3. C3D: первая «чистая» 3D CNN Tran et al., ICCV 2015

C3D = Convolutional 3D

Tran, Bourdev, Fergus, Torresani, Paluri (Facebook AI, Dartmouth), *Learning Spatiotemporal Features with 3D Convolutional Networks*. Сеть в стиле VGG, где **все** свёртки — $3 \times 3 \times 3$: 8 conv, 5 pooling, 2 fc. Обучена с нуля на Sports-1M (1.1 млн видео, 487 классов).



Как использовали

Выход fc6 (4096 чисел) на клип из 16 кадров — универсальный дескриптор видео: признаки клипов усредняют и учат линейный SVM (UCF101 $\approx$ 85%).

Вывод статьи: ядро $3 \times 3 \times 3$ лучше, чем $1 \times 3 \times 3$ и более длинные по времени. Минус: 3D-сеть с нуля требует много данных и compute — отсюда I3D и R(2+1)D.

3. Цена третьего измерения

MAC-операции одного слоя

$$\text{MACs}_{3D} = T'H'W' \cdot C_{\text{out}}C_{\text{in}} \cdot k_t k^2, \quad \text{MACs}_{2D} = T'H'W' \cdot C_{\text{out}}C_{\text{in}} \cdot k^2$$
$$\Rightarrow \text{MACs}_{3D}/\text{MACs}_{2D} = k_t \text{ (то же для числа параметров)}$$

Пример: слой conv3a из C3D

- ▶ вход $128 \times 16 \times 28 \times 28$, $C_{\text{out}} = 256$, ядро $3 \times 3 \times 3$, stride 1
- ▶ позиций: $16 \cdot 28 \cdot 28 = 12544$; рецептивное поле на выход: $3 \cdot 3 \cdot 3 \cdot 128 = 3456$
- ▶ 3D: $12544 \cdot 256 \cdot 3456 \approx \mathbf{11.1}$ GMAC 2D (3×3): $12544 \cdot 256 \cdot 1152 \approx \mathbf{3.7}$ GMAC

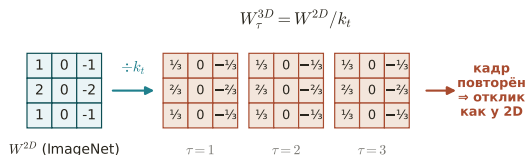
При $k_t = 3$ слой втрое дороже и по MAC, и по параметрам. Осторожно с «GFLOPs»: одни авторы считают MAC за 1 операцию, другие — за 2.

3. I3D: инфляция 2D-весов

Carreira, Zisserman, 2017

I3D = Inflated 3D ConvNet

2D-сеть **Inception-v1 (BN-Inception)** с весами ImageNet, у которой все ядра и pooling «раздуты» по времени: $k \times k \rightarrow k_t \times k \times k$. Используется в **Two-Stream I3D**: одна I3D на RGB, другая на оптическом потоке, logits усредняются.



Инфляция и зачем делить на k_t

$$W_{::,\tau,::}^{3D} = \frac{1}{k_t} W^{2D}, \quad \tau = 1, \dots, k_t$$

Статичное видео

$$x_1 = \dots = x_{k_t} = x:$$

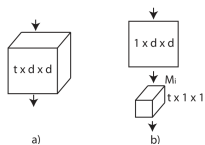
$$\sum_{\tau} \frac{1}{k_t} W^{2D} * x = W^{2D} * x$$

Интуиция. В начале обучения I3D ведёт себя как ImageNet-сеть на повторённом кадре; различия между кадрами она учит при дообучении.

3. R(2+1)D: две свёртки вместо кубика Tran et al., 2018

R(2+1)D = ResNet с (2+1)D-блоками

Tran, Wang, Torresani, LeCun, Paluri (Facebook AI), *A Closer Look at Spatiotemporal Convolutions for Action Recognition*, CVPR 2018. Берут 3D-ResNet (R3D-18/34) и каждую свёртку $t \times d \times d$ заменяют парой «2D по кадру + 1D по времени». В статье сравнили R2D, MCx, rMCx, R3D и R(2+1)D — последняя лучшая на Kinetics и Sports-1M.



Tran et al., CVPR 2018,
Fig. 2

$$t \times d \times d \rightarrow (1 \times d \times d) + (t \times 1 \times 1)$$

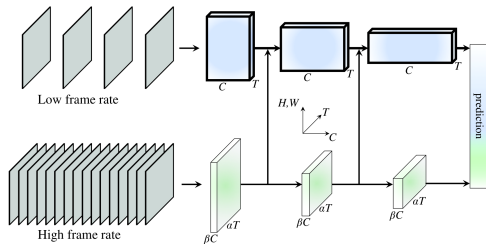
$$P_{3D} = td^2N_{i-1}N_i, \quad P_{(2+1)D} = d^2N_{i-1}M_i + tM_iN_i$$

$$\text{Равная ёмкость: } M_i = \frac{td^2N_{i-1}N_i}{d^2N_{i-1} + tN_i}$$

$$Y = \text{Conv}_{t \times 1 \times 1}(\sigma(\text{Conv}_{1 \times d \times d}(X)))$$

Выигрыш не в FLOPs (они равны), а в оптимизации: вдвое больше ReLU, пространство и время учатся раздельно.

3. SlowFast: две частоты Feichtenhofer et al., 2019



Feichtenhofer et al., ICCV 2019, Fig. 1

Два пути

Slow: T кадров, C каналов — семантика. **Fast:** αT кадров, βC каналов, $\beta \approx 1/8$ — быстрое движение. Lateral connections: Fast $\rightarrow$ Slow.

Интуиция. Движению нужна высокая частота кадров, но не много каналов; семантике — много каналов, но её можно смотреть реже. Это two-stream без внешнего оптического потока.

3. Листинг: SlowFast

Алгоритм: SlowFast (ResNet-50, $T \times \tau = 4 \times 16$)

вход: ролик V ; $T = 4$ кадра Slow, шаг $\tau = 16$; $\alpha = 8$, $\beta = 1/8$

выход: логиты классов s

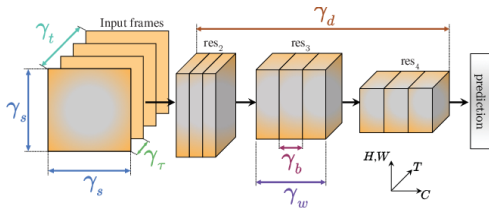
1. Slow-вход: T кадров с шагом τ (4 кадра за ≈ 2 с)
2. Fast-вход: αT кадров с шагом τ/α (32 кадра, шаг 2)
3. **для** стадий $\text{res}_2, \dots, \text{res}_5$:
 - 3.1 $h_{\text{slow}} \leftarrow \text{Block}_{\text{slow}}(h_{\text{slow}})$ — C каналов; временные ядра $k_t > 1$ только в $\text{res}_4, \text{res}_5$
 - 3.2 $h_{\text{fast}} \leftarrow \text{Block}_{\text{fast}}(h_{\text{fast}})$ — βC каналов; временные ядра на всех стадиях
 - 3.3 lateral: $h_{\text{slow}} \leftarrow [h_{\text{slow}}; \text{Conv}_{5 \times 1 \times 1, \text{stride } \alpha}(h_{\text{fast}})]$
4. $z \leftarrow [\text{GAP}(h_{\text{slow}}); \text{GAP}(h_{\text{fast}})]$, $s \leftarrow \text{FC}(z)$

Time-strided свёртка в lateral сжимает время Fast в α раз, чтобы совпасть с Slow. Связи только Fast $\rightarrow$ Slow. Fast-путь даёт $\approx 20\%$ вычислений при α раз большей частоте кадров.

3. X3D: рост по одной оси Feichtenhofer, CVPR 2020

X3D = eXpand 3D

Feichtenhofer (Facebook AI), *X3D: Expanding Architectures for Efficient Video Recognition*. Старт — крошечная 2D-сеть X2D (ResNet с channel-wise separable свёртками, как MobileNet). Её пошагово «раздувают» по шести осям, каждый раз выбирая одну лучшую.



Feichtenhofer, CVPR 2020, Fig. 1

γ_t — число кадров, γ_τ — шаг по времени, γ_s — разрешение, γ_w — ширина, γ_b — ширина bottleneck, γ_d — глубина. Итог: семейство X3D-XS, S, M, L, XL.

Интуиция. Не масштабировать всё сразу, а добавлять compute туда, где он даёт больше точности. X3D-M: 76% на Kinetics при ≈ 6 GFLOPs на view — в разы дешевле SlowFast.

3. Листинг: X3D

Алгоритм: X3D (coordinate descent по осям)

вход: базовая сеть X2D; оси $\mathcal{A} = \{\gamma_t, \gamma_\tau, \gamma_s, \gamma_w, \gamma_b, \gamma_d\}$; целевой бюджет FLOPs

выход: семейство моделей X3D-XS $\rightarrow$ XL

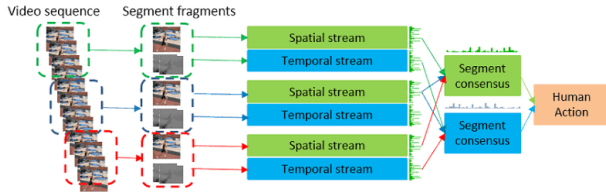
1. $M \leftarrow \text{X2D}$
2. **пока** $\text{FLOPs}(M)$ меньше бюджета:
 - 2.1 **для** каждой оси $a \in \mathcal{A}$: $M_a \leftarrow$ расширить M по a так, чтобы $\text{FLOPs}(M_a) \approx 2 \cdot \text{FLOPs}(M)$; обучить, измерить точность
 - 2.2 $a^* \leftarrow \arg \max_a \frac{\text{Acc}(M_a) - \text{Acc}(M)}{\text{FLOPs}(M_a) - \text{FLOPs}(M)}$
 - 2.3 $M \leftarrow M_{a^*}$; сохранить M как очередную модель семейства
3. **вернуть** все сохранённые M

На каждом шаге расширяется ровно одна ось. Вывод статьи: для видео выгодны высокое пространственно-временное разрешение и тонкие каналы — общую ширину сети растить почти не нужно.

3. TSN: покрыть весь ролик Wang et al., ECCV 2016

TSN = Temporal Segment Networks

Wang, Xiong, Wang, Qiao, Lin, Tang, Van Gool, *Temporal Segment Networks: Towards Good Practices for Deep Action Recognition*. Видео делят на K сегментов, из каждого берут короткий фрагмент T_k (RGB-кадр и стопка flow), прогоняют общей two-stream сетью (BN-Inception) и объединяют ответы: $\text{TSN}(T_1, \dots, T_K) = \mathcal{H}(\mathcal{G}(F(T_1; W), \dots, F(T_K; W)))$.



Wang et al., ECCV 2016, Fig. 1

Интуиция. Редкие кадры по всей длине дешевле плотного клипа и видят весь ролик. Consensus $\mathcal{G}$ — среднее, поэтому порядок сегментов не учитывается: локальной динамики нет. Хороший бейзлайн при сильном appearance bias.

3. Листинг: TSN

Алгоритм: TSN

вход: видео V , метка y ; K сегментов; shared backbone $F(\cdot; W)$

выход: video-level class distribution $p(y \mid V)$

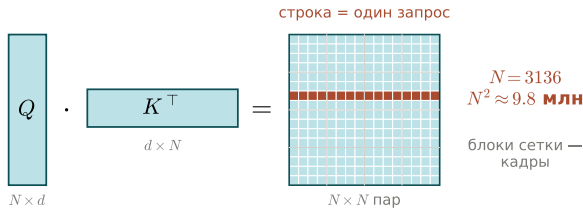
1. Разбить V на K равных временных сегментов $S_1, \dots, S_K$.
2. **Для** $k = 1, \dots, K$:
 - 2.1 $t_k \sim \text{Uniform}(S_k)$;
 - 2.2 $T_k \leftarrow$ snippet в момент t_k (RGB: 1 кадр; flow: стек L flow-полей);
 - 2.3 $s_k \leftarrow F(T_k; W) \in \mathbb{R}^C$.
3. Video-level consensus: $\bar{s} \leftarrow \frac{1}{K} \sum_k s_k$.
4. Предсказание и loss: $p \leftarrow \text{softmax}(\bar{s})$; $\mathcal{L} \leftarrow \text{CE}(y, p)$.

Loss считается по консенсусу, а не по кадру: сеть учится на уровне всего видео, видя лишь K кадров.

4. Self-attention

Scaled dot-product attention, $X \in \mathbb{R}^{N \times d}$

$$Q = XW_Q, K = XW_K, V = XW_V, \quad \text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



QK^T — матрица $N \times N$, сходство каждой пары токенов. Деление на $\sqrt{d_k}$: иначе скалярные произведения растут с размерностью, softmax насыщается, градиенты портятся.

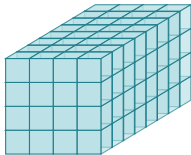
Интуиция. Каждый токен одной строкой матрицы решает, на какие токены всего клипа смотреть. Цена — N^2 пар.

4. Токенизация видео

Пространственно-временной патч $t \times p \times p$ (в статьях — tubelet)

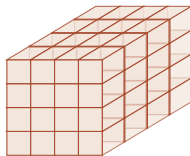
$$N = \frac{T}{t} \cdot \frac{H}{p} \cdot \frac{W}{p}. \quad \text{Пример: } T = 16, H = W = 224, p = 16.$$

патч $1 \times p \times p$



$16 \cdot 14 \cdot 14 = 3136$ токенов

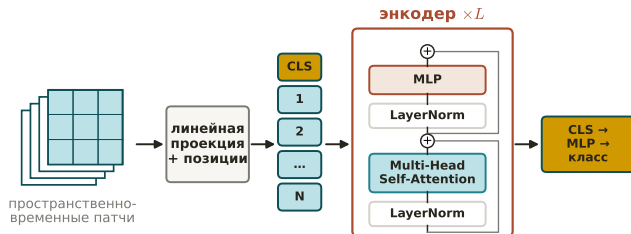
пространственно-временной патч $2 \times p \times p$



$8 \cdot 14 \cdot 14 = 1568$ токенов

Вдвое меньше токенов — вчетверо меньше пар: $(1568)^2 \approx 2.5$ млн вместо $(3136)^2 \approx 9.8$ млн. Плата — грубое разрешение по времени.

4. Видео-трансформер: стандартная схема (ViT)



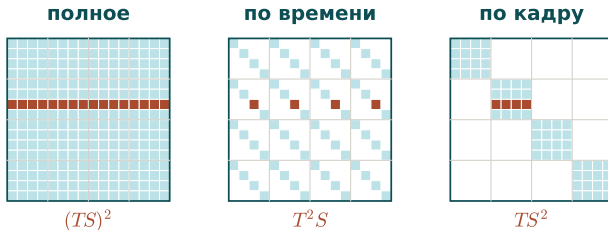
Pre-LN блок энкодера

$$u = z + \text{MHSA}(\text{LN}(z)), \quad z' = u + \text{MLP}(\text{LN}(u))$$

Пространственно-временной патч → вектор линейной проекцией + позиционные эмбеддинги (время и пространство); обучаемый CLS после L блоков идёт в классификатор.

Всё видео-специфичное — в токенизации и в том, *какие пары токенов* пускают в attention. ViViT, TimeSformer, Swin и MViT меняют именно это.

4. Joint vs divided attention



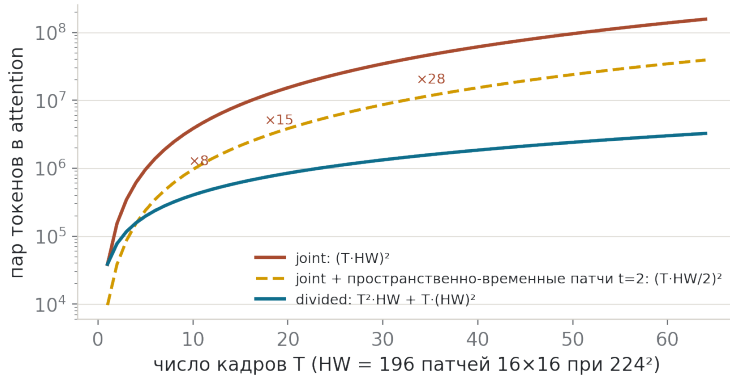
$S = HW$ патчей на кадр; $T = 16$, $S = 196$

Joint: $(TS)^2 = T^2 S^2 \approx 9.8$ млн пар. Divided:
 $T^2 S + TS^2 \approx 50$ тыс. + 615 тыс. ≈ 0.66 млн пар — в ≈ 15 раз меньше.

Divided: сначала токен смотрит на тот же патч во всех кадрах (время), затем на все патчи своего кадра (пространство). Удвоили T : joint растёт в 4 раза, пространственная часть divided — только в 2.

4. Как растёт цена с длиной клипа

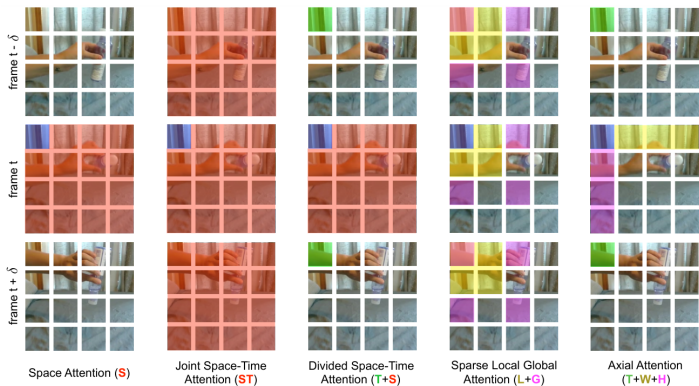
Полное внимание растёт как T^2 , факторизованное — почти линейно



Логарифмическая шкала. Пространственно-временные патчи ($t = 2$) сдвигают кривую joint вниз в 4 раза, но не меняют её наклон; факторизация меняет сам закон роста. Чем длиннее клип, тем больше выигрыш.

4. TimeSformer

Bertasius, Wang, Torresani, 2021



Bertasius, Wang, Torresani, ICML 2021, Fig. 2

Синий — query-патч, цветные — куда он смотрит. Лучшим в статье оказался вариант Divided Space-Time (T+S): два attention-прохода в каждом блоке.

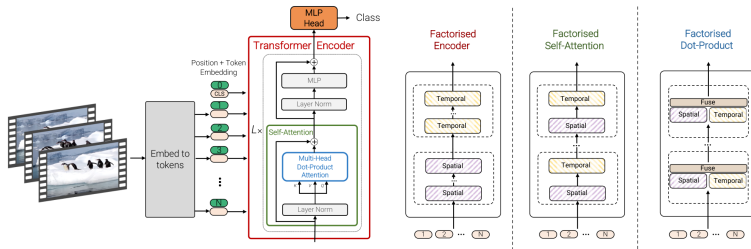
4. TimeSformer: куда смотрит внимание



Bertasius, Wang, Torresani, ICML 2021

Карты внимания подсвечивают руки и предметы, с которыми работают, а не фон. Это качественный, а не количественный аргумент, что модель использует время.

4. ViViT Arnab et al., 2021



Arnab et al., ICCV 2021

ViViT — семейство альтернативных архитектур

Model 1: полное joint space-time attention — дорогой baseline, $O((TS)^2)$.

Model 2: factorised encoder. **Model 3:** factorised self-attention. **Model 4:** factorised dot-product attention.

Важно: Models 2-4 не образуют pipeline — для конкретной модели выбирают один вариант. Покадровые патчи или tubelet embedding — это способ построить входные токены, а не выбор Model 2/3/4.

4. ViViT Model 2: Factorised Encoder

Сначала spatial ViT кодирует каждый кадр и оставляет от него один CLS, затем второй трансформер учит связи между кадрами:

видео $\rightarrow$ ViT на каждом кадре $\rightarrow T$ frame embeddings $\rightarrow$ temporal ViT $\rightarrow$ класс.

Алгоритм: ViViT Model 2 — Factorised Encoder

вход: $X \in \mathbb{R}^{T \times S \times d}$

выход: логиты классов

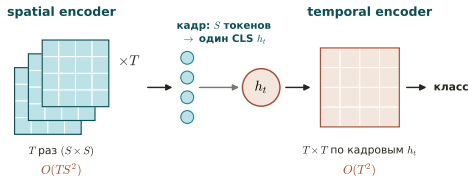
1. Spatial encoder: **для** каждого кадра $t = 1..T$: добавить CLS-токен c_t ;
 $H_t \leftarrow \text{SpatialViT}([c_t, X_{t,1}, \dots, X_{t,S}]); z_t \leftarrow H_{t,\text{CLS}}$. Итог: $Z = [z_1, \dots, z_T] \in \mathbb{R}^{T \times d}$.
2. Temporal encoder: $h_{\text{video}} \leftarrow \text{TemporalViT}([\text{CLS}_{\text{video}}, z_1, \dots, z_T])$.
3. Классификация: $\text{logits} \leftarrow \text{MLPHead}(h_{\text{video}}[\text{CLS}_{\text{video}}])$.

Формально

$h_t = \text{SpatialEncoder}([\text{CLS}_t, z_{t,1}, \dots, z_{t,S}])_{\text{CLS}}$,

$h_{\text{video}} = \text{TemporalEncoder}([\text{CLS}_{\text{video}}, h_1, \dots, h_T])_{\text{CLS}}$

4. ViViT Model 2: цена и компромисс



Стоимость

Spatial encoder на T кадрах: $O(TS^2)$; temporal encoder на T эмбедингах: $O(T^2)$; итого $O(TS^2 + T^2)$. Одной матрицы $TS \times TS$ здесь нет: temporal attention идёт между кадрами, а не между траекториями патчей.

Плюсы

- ▶ понятная декомпозиция: «что на кадре», потом «как кадры связаны»
- ▶ дешёвый temporal encoder — только T токенов
- ▶ хорошо переносит pretrained image-ViT

Минусы

- ▶ до temporal encoder кадр сжат из S токенов в один CLS
- ▶ мелкая локальная динамика теряется или должна уместиться в h_t
- ▶ нет отдельной временной траектории для каждого патча

4. ViViT Model 3: Factorised Self-Attention

Аналог divided attention из TimeSformer: внутри *каждого* блока сначала spatial attention, затем temporal. В отличие от Model 2, кадр не сжимается до одного CLS.

Алгоритм: ViViT Model 3 — Factorised Self-Attention

вход: $X \in \mathbb{R}^{T \times S \times d}$

выход: логиты классов

Для каждого блока трансформера (выход: CLS / pooling $\rightarrow$ MLP head):

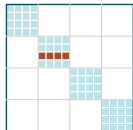
1. Spatial: **для** каждого кадра t : $X_{t,:,:} \leftarrow \text{SpatialMSA}(X_{t,:,:})$; связи $(t, s) \rightarrow \{(t, s') : s' = 1..S\}$.
2. Temporal: **для** каждой позиции s : $X_{:,s,:} \leftarrow \text{TemporalMSA}(X_{:,s,:})$; связи $(t, s) \rightarrow \{(t', s) : t' = 1..T\}$.
3. MLP + residual connections.

Формально и цена

$$\tilde{X}_{t,:,:} = \text{SpatialMSA}(X_{t,:,:}), \quad X_{:,s,:}^{\text{next}} = \text{TemporalMSA}(\tilde{X}_{:,s,:}), \quad \text{цена } O(TS^2 + ST^2)$$

4. ViViT Model 3: те же матрицы, что у TimeSformer

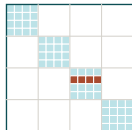
шаг 1: spatial MSA



T блоков $S \times S$

затем
→

шаг 2: temporal MSA



S блоков $T \times T$

в каждом
блоке $\times L$

все $T \cdot S$
токенов
сохранены

Справа токены упорядочены по позиции s , поэтому временные связи видны как S диагональных блоков $T \times T$. ViViT Model 3 — *альтернатива* Model 2: здесь нет сжатия кадра $S \rightarrow 1$ перед temporal attention.

Плюсы

- ▶ сохраняет все $T \times S$ токенов между шагами
- ▶ учит временную траекторию для каждой позиции
- ▶ более тонкое моделирование времени, чем Model 2

Минусы

- ▶ дороже Model 2: $O(TS^2 + ST^2)$ вместо $O(TS^2 + T^2)$
- ▶ два attention-модуля в блоке: больше параметров и latency
- ▶ разные позиции разных кадров связаны только через два шага

4. ViViT Model 4: Factorised Dot-Product Attention

В одном Multi-Head Attention часть голов смотрит только по пространству, часть — только по времени. Ветки работают *параллельно*, выходы конкатенируются.

Алгоритм: ViViT Model 4 — Factorised Dot-Product Attention

вход: $X \in \mathbb{R}^{T \times S \times d}$; h голов attention

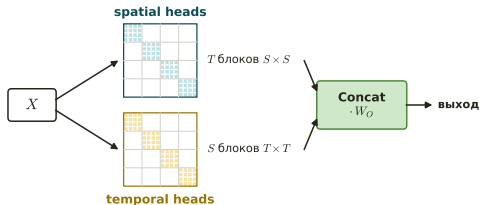
выход: логиты классов

1. Разделить головы: H_{space} — для пространства, H_{time} — для времени.
2. Каждая spatial-голова: $(t, s) \rightarrow \{(t, s') : s' = 1..S\}$.
3. Каждая temporal-голова: $(t, s) \rightarrow \{(t', s) : t' = 1..T\}$.
4. Объединить: $Y \leftarrow \text{Concat}(Y_{\text{space}}, Y_{\text{time}}) W_O$.
5. MLP + residual connections.

Формально и цена

$Y_{\text{space}} = \text{MSA}_{\text{space}}(X)$, $Y_{\text{time}} = \text{MSA}_{\text{time}}(X)$, $Y = \text{Concat}(Y_{\text{space}}, Y_{\text{time}}) W_O$. Связей $O(TS^2 + ST^2)$; параметров — как у МНА: головы делятся между двумя типами.

4. ViViT Model 4: параллельные ветки



Model 3: spatial $\rightarrow$ temporal *последовательно*. Model 4: spatial $\parallel$ temporal *параллельно*, смешивание — в $\text{Concat} \cdot W_O$.

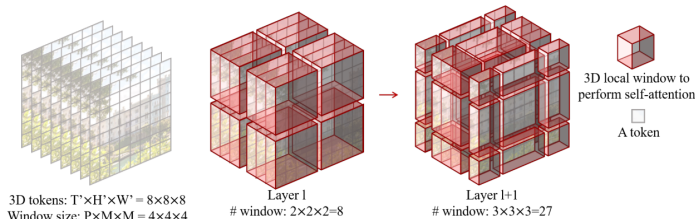
Плюсы

- ▶ одна MHA-операция вместо двух последовательных MSA
- ▶ параметров примерно как у обычного блока
- ▶ spatial и temporal связи есть в каждом блоке

Минусы

- ▶ каждая ветвь получает только часть голов
- ▶ temporal-головы не видят spatial-обогащённые признаки того же блока, и наоборот
- ▶ space и time смешиваются поздно — после $\text{Concat} \cdot W_O$

4. Video Swin Transformer Liu et al., 2022



Liu et al., CVPR 2022, Fig. 3

Локальные 3D-окна вместо global и factorised attention

Inductive bias: близкие токены обычно важнее дальних, поэтому attention считают только внутри небольшого 3D-окна $M_t \times M_h \times M_w$. Пример: $2 \times 7 \times 7 \Rightarrow M = 98$ токенов в окне (в статье по умолчанию $8 \times 7 \times 7$). Каждый токен видит M токенов, а не все TS .

Следующий блок сдвигает разбиение окон — токены из разных окон начинают взаимодействовать. Это shifted windows из Swin, перенесённые на 3D, плюс иерархическая архитектура.

4. Листинг: Video Swin Transformer

Алгоритм: Video Swin — пара последовательных блоков

вход: $X \in \mathbb{R}^{T \times H_p \times W_p \times d}$; окно $M_t \times M_h \times M_w$

выход: признаки следующей стадии

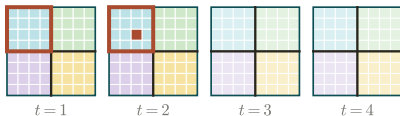
1. **W-MSA:** разбить X на непересекающиеся окна; $Y_w \leftarrow \text{MSA}(X_w)$ для каждого окна.
2. **SW-MSA:** циклически сдвинуть решётку на $(M_t/2, M_h/2, M_w/2)$; снова разбить на окна того же размера и считать MSA внутри них; attention mask не смешивает края, склеенные cyclic shift.
3. **Patch merging** между стадиями: H_p, W_p уменьшить примерно в 2 раза, каналы увеличить.

Одно окно w и цена

$\text{W-MSA}(X_w) = \text{softmax}(Q_w K_w^\top / \sqrt{d_h} + B) V_w$, $X_w \in \mathbb{R}^{M \times d}$, $M = M_t M_h M_w$; B — относительное позиционное смещение. Окон $n_{\text{win}} = TS/M$; цена $n_{\text{win}} \cdot M^2 = TS \cdot M \Rightarrow$ при фиксированном M это $O(TS)$ — почти линейно по числу токенов.

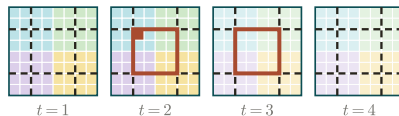
4. Video Swin: окна и их сдвиг

блок ℓ : W-MSA, окно $2 \times 4 \times 4$



токен видит только своё 3D-окно

блок $\ell+1$: SW-MSA, сдвиг $(1, 2, 2)$

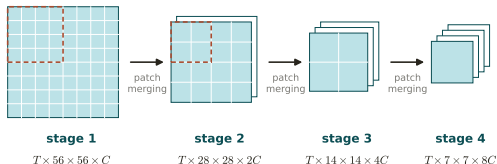


новое окно пересекает 4 старых окна

Для простоты $T = 4$, $H_p = W_p = 8$, окно $2 \times 4 \times 4$, сдвиг $(\Delta_t, \Delta_h, \Delta_w) = (1, 2, 2)$. Цвет — окно блока ℓ (бледные — кадры 3-4), красный — один токен.

W-MSA: только локальные связи внутри окна. SW-MSA: тот же токен попадает в окно, которое пересекает прежние границы по h , w и времени. Так через следующий блок связываются соседние окна. В матричном виде оба шага — блочно-диагональные матрицы с блоками $M \times M$, но с разной группировкой токенов.

4. Video Swin: иерархия как у CNN



Patch merging на границах стадий сжимает H, W вдвое и удваивает каналы; время обычно сохраняется дольше — точный schedule зависит от конфигурации. Пунктир — окно.

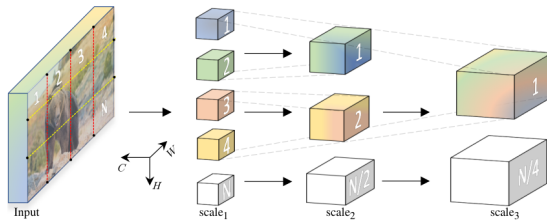
Плюсы

- ▶ локальный attention почти линеен: $O(TS \cdot M_t M_h M_w)$, а не $O((TS)^2)$
- ▶ shifted windows дают связи между окнами без global attention
- ▶ пирамида удобна для detection, segmentation и других dense-задач

Минусы

- ▶ один блок видит только локальный контекст
- ▶ глобальные связи распространяются постепенно через глубину
- ▶ shift + attention mask усложняют реализацию и оптимизацию ядер
- ▶ жёсткая локальность слабее для дальних связей

4. MViT: pooling attention Fan et al., 2021



Fan et al., ICCV 2021, Fig. 1

Другой ответ, чем у Video Swin

Video Swin сохраняет много токенов, но даёт каждому только локальный контекст. MViT оставляет attention более глобальным, но *уменьшает число queries, keys и values* через pooling и строит multiscale-пирамиду.

Pooling Q задаёт разрешение выхода стадии, pooling K, V сокращает контекст, с которым сравниваются запросы. Отсюда иерархия: «много токенов, мало каналов» → «мало токенов, много каналов».

4. Листинг: MViT, Multi-Head Pooling Attention

Алгоритм: МНРА

вход: $X \in \mathbb{R}^{T \times H_p \times W_p \times d}$; после flatten $X \in \mathbb{R}^{N \times d}$, $N = TH_pW_p$

выход: признаки стадии

1. Линейные проекции: $\hat{Q} = XW_Q$, $\hat{K} = XW_K$, $\hat{V} = XW_V$.
2. Pooling в 3D: $Q = P_Q(\hat{Q})$, $K = P_K(\hat{K})$, $V = P_V(\hat{V})$ — по времени и пространству, возможно с разными stride.
3. Pooling attention: $A = \text{softmax}(QK^\top / \sqrt{d_h})$, $Y = AV$.
4. Residual + MLP.
5. На границе стадии: query pooling уменьшает $T \times H_p \times W_p$, размерность каналов d растёт.

Формально и цена

$\text{МНРА}(X) = \text{softmax}(QK^\top / \sqrt{d_h})V$, $Q \in \mathbb{R}^{N_q \times d_h}$, $K, V \in \mathbb{R}^{N_k \times d_h} \Rightarrow A \in \mathbb{R}^{N_q \times N_k}$, цена $O(N_q N_k)$.
Это точнее, чем $O(N^2)$: число queries и keys у MViT разное.

4. MViT: прямоугольная матрица внимания

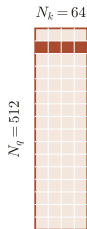
dense Q : $8 \times 8 \times 8 = 512$ токенов (показаны 4 кадра)



кубик $2 \times 2 \times 2$
 $\rightarrow$ 1 токен



pooled K, V : $4 \times 4 \times 4 = 64$ токена



$$A = QK^T \in \mathbb{R}^{512 \times 64}$$

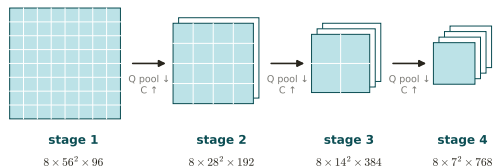
$$512 \cdot 64 = 32768$$

вместо $512^2 = 262144$

в 8 раз дешевле

Решётка $8 \times 8 \times 8 = 512$ токенов. Q оставляем полным ($N_q = 512$), K, V пулим со stride $2 \times 2 \times 2$: $N_k = 512/8 = 64$. Строки — queries, столбцы — более редкие keys/values. $N_k \ll N_q \Rightarrow \text{цена} \ll O(N^2)$.

4. MViT: multiscale-пирамида



Пример MViT-B: время (8) не сжимается, H, W — вдвое на каждой стадии, каналы удваиваются. В других конфигурациях schedule pooling по T, H, W может быть другим.

Плюсы

- ▶ CNN-подобная иерархия: глубже — меньше токенов, больше каналов
- ▶ pooling K, V снижает цену до $O(N_q N_k)$, $N_k \ll N_q$
- ▶ пирамида признаков подходит и для dense-задач
- ▶ широкий, почти глобальный контекст на грубом масштабе

Минусы

- ▶ pooling необратимо теряет мелкие детали по пространству и времени
- ▶ schedule pooling для Q, K, V нужно подбирать
- ▶ архитектура и residual path сложнее обычного ViT
- ▶ ранние стадии высокого разрешения всё ещё дорогие

4. Видео-трансформеры: кто на чём экономит

Модель	Что ограничивает или сжимает	Цена attention	Как строится дальняя связь
TimeSformer	факторизует связи по осям времени и пространства	$T^2S + TS^2$	temporal шаг + spatial шаг
ViViT Model 2	сжимает каждый кадр до одного CLS перед temporal encoder	$TS^2 + T^2$	через кадровые эмбединги
ViViT Model 3	факторизует MSA внутри каждого блока	$TS^2 + ST^2$	два последовательных attention-шага
ViViT Model 4	делит головы на spatial и temporal	$TS^2 + ST^2$	через слияние выходов голов
Video Swin	ограничивает attention фиксированным 3D-окном	$O(TS \cdot M)$	сдвиг окон через глубину
MViT	pooling токенов и K, V	$O(N_q N_k)$	грубые multiscale-токены

TimeSformer и ViViT факторизуют attention по осям. Video Swin смотрит недалеко за один шаг, но сдвигает окна между слоями. MViT оставляет attention глобальнее, но сокращает сами последовательности, особенно keys/values. Три разных ответа на один вопрос: как не платить $O((THW)^2)$.

5. Почему видео удобно для SSL

Разметка дорогая

- ▶ действие и его границы во времени
- ▶ объекты, взаимодействия, треки

Структура бесплатна

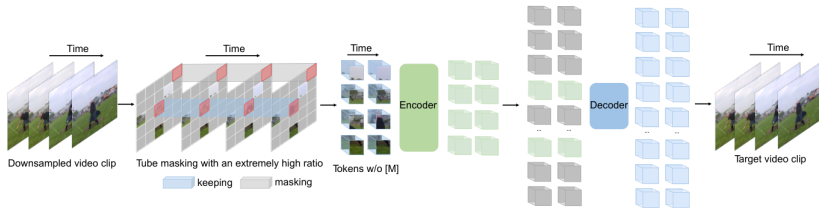
- ▶ соседние кадры коррелированы
- ▶ объекты сохраняют идентичность
- ▶ движение ограничивает будущее
- ▶ звук и субтитры — ещё сигналы

Схема

Предобучение на огромном неразмеченном корпусе → перенос: linear probing (энкодер заморожен) или full fine-tuning на маленьком размеченном наборе.

Три типа цели: восстановить пиксели (VideoMAE), различить видео (контраст), предсказать признаки скрытого (V-JEPA).

5. VideoMAE Tong et al., 2022



Tong et al., NeurIPS 2022, Fig. 1

Маскирование 90-95% пространственно-временных патчей

M — скрытые токены, X_p — пиксели токена p (нормированы внутри патча): $\mathcal{L}_{\text{MAE}} = \frac{1}{|M|} \sum_{p \in M} \|\hat{X}_p - X_p\|_2^2$

Асимметрия: тяжёлый энкодер видит только видимые токены, лёгкий декодер восстанавливает пиксели. После предобучения декодер выбрасывают, энкодер дообучают.

5. VideoMAE: большая маска-трубка

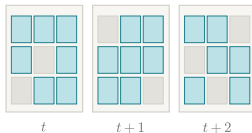
Цена энкодера

Видимая доля r : attention
 $\propto (rN)^2$
 $r = 0.1$: в $1/0.1^2 = 100$ раз
дешевле

Tube masking

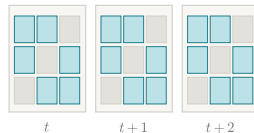
Одна маска на все кадры:
 $m_{\tau,ij} = m_{ij} \quad \forall \tau$

случайная маска



дырку можно списать с соседнего кадра

маска-трубка



$m_{\tau,ij} = m_{ij}$: списать неоткуда

Интуиция. Видео избыточно: при маленькой или случайной маске скрытый патч виден в соседнем кадре, и модель просто копирует. Трубка закрывает эту лазейку — приходится понимать сцену.

5. Контрастное обучение и InternVideo2

InfoNCE

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(z, z^+)/\tau)}{\exp(\text{sim}(z, z^+)/\tau) + \sum_{z^-} \exp(\text{sim}(z, z^-)/\tau)}$$

z, z^+ — два view одного видео, z^- — другие видео из батча, τ — температура.

Модель учит признаки, инвариантные к аугментациям. Для видео опасно выбрать аугментации, которые ломают само движение или порядок кадров.

InternVideo2 (Wang et al., 2024)

Masked modeling + контраст видео-текст-аудио + next-token цели.
Foundation-модель для video-LLM, а не только классификатор.
Сравнивая числа, учитывайте масштаб: параметры, данные, текст/аудио, frozen или fine-tune.

5. V-JEPA: признаки вместо пикселей

Bardes et al., 2024

Цель

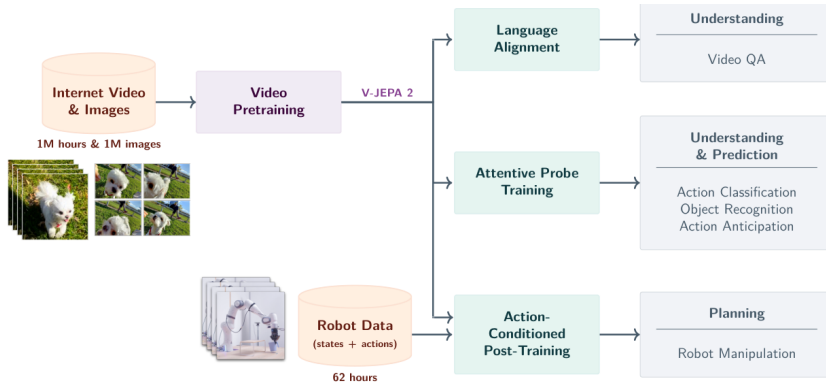
$$\mathcal{L}_{\text{JEPA}} = \frac{1}{|M|} \sum_{y \in M} \left\| P_{\phi}(E_{\theta}(X_{\text{visible}}), \Delta_y) - \text{sg}(\bar{E}(X)_y) \right\|_1$$

EMA target encoder: $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$, $\tau \approx 0.999$

- ▶ E_{θ} — context encoder по видимым токенам; P_{ϕ} — predictor; Δ_y — позиция цели
- ▶ $\bar{E}$ — target encoder: без градиента (sg = stop-gradient), веса — EMA от E_{θ}

Интуиция. Пиксельная цель заставляет тратить ёмкость на текстуры, шум и освещение. Предсказание в пространстве признаков учит структуру сцены. Риск — collapse (все эмбединги одинаковы); от него спасают EMA и stop-gradient.

5. V-JEPA 2: один энкодер — много задач



Assran et al., V-JEPA 2, 2025, Fig. 1

V-JEPA 2 масштабирует идею на большие видеокорпуса: понимание, предсказание, планирование действий робота. V-JEPA 2.1 добавляет плотные сигналы для depth и segmentation.

6. Датасеты

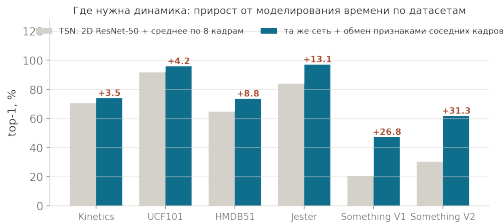
Датасет	Что содержит	Что проверяет	Ограничение
UCF101 / HMDB51	короткие клипы действий	быстрый fine-tuning, мало данных	насыщены, сильный appearance bias
Kinetics-400/600/700	клипы YouTube	масштабное распознавание действий	класс часто виден по кадру; видео исчезают
Something- Something v2	шаблонные действия рук с предметами	направление движения, взаимодействие	фон и объекты почти не помогают
HowTo100M	видео + ASR-субтитры	видео-языковое предобучение	шумный, плохо синхронный текст
Ego4D	длинное эгоцентричное видео	память, прогноз, рука-предмет	дорогая разметка, тряска камеры

Kinetics отвечает на «что за сцена», Something-Something — на «что сделали и в какую сторону».

6. Appearance bias: балл без движения

Диагностика

$\Delta_{\text{temporal}} = \text{Acc}_{\text{video}} - \text{Acc}_{\text{single-frame}}$. Маленькое $\Delta \Rightarrow$ задачу решают по внешнему виду.



числа из Lin, Gan, Han, ICCV 2019, Table 1

TSN — Temporal Segment Network (Wang et al., 2016): 2D CNN на K кадрах из разных сегментов, logits усредняются; порядок кадров не учитывается.

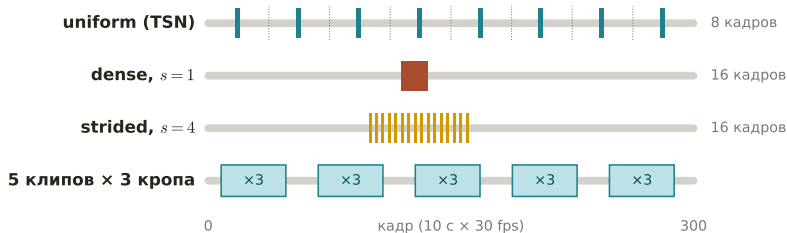
Вторая сеть — та же, но в каждом блоке $\approx 1/4$ каналов сдвигается к соседним кадрам $t \pm 1$; 0 доп. FLOPs и параметров.

Сети отличаются только обменом признаками между соседними кадрами. На Kinetics и UCF101 время даёт +3–4 п.п., на Something-Something — около +30 п.п.

6. Сэмплирование кадров

Strided clip: T кадров с шагом s

$(x_{t_0}, x_{t_0+s}, x_{t_0+2s}, \dots, x_{t_0+(T-1)s})$ охват $\approx (T-1)s$ кадров при тех же T .



Малый s — подробное короткое движение. Большой s — длинный охват, быстрый жест может выпасть (алиасинг). Uniform (TSN) покрывает ролик, но локального движения не видит.

6. Test-time views и честная цена

Усреднение по V views (клипы × кропы)

Среднее логитов: $\bar{s}_i = \frac{1}{V} \sum_{v=1}^V s_{i,v}$, $\hat{y}_i = \arg \max_c \bar{s}_{i,c}$

Среднее вероятностей: $\bar{p}_i = \frac{1}{V} \sum_{v=1}^V \text{softmax}(s_{i,v})$

softmax нелинеен, поэтому два способа дают разные ответы — протокол надо указывать.

Главное правило

Total compute $\approx$ GFLOPs per view $\times$ number of views

50 GFLOPs на view $\times$ 30 views $\approx$ 1500 GFLOPs на видео. Сравнивать модели по GFLOPs на один view без протокола теста нечестно.

6. Метрики

Три accuracy

$$\text{Acc@1} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i], \quad \text{Acc@5} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i \in \text{Top5}(s_i)],$$

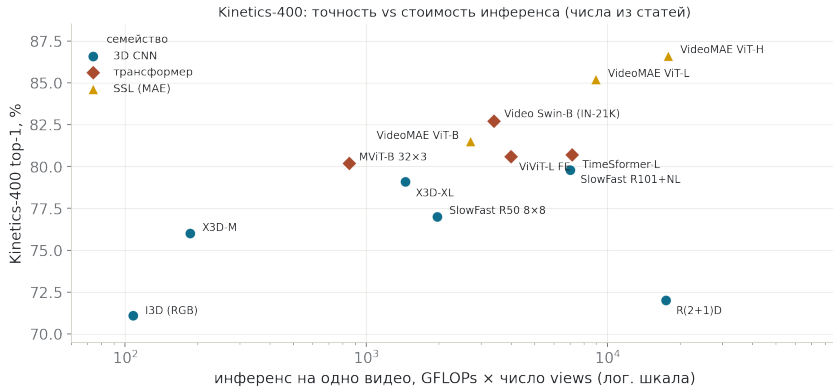
$$\text{MacroAcc} = \frac{1}{C} \sum_{c=1}^C \frac{1}{N_c} \sum_{i: y_i=c} \mathbf{1}[\hat{y}_i = c]$$

Топ-5 прощает путаницу близких классов, но скрывает ошибку, если нужен один ответ. Macro не даёт частым классам заглушить редкие.

Для production дополнительно: latency на фиксированном железе, throughput и память, качество при 1 view, калибровка, метрики по классам и срезам, устойчивость к blur, сжатию, смене fps.

+0.5 п.п. ценой 30 views часто хуже чуть более слабой модели, которая укладывается в бюджет задержки на одном клипе.

6. Pareto: качество против стоимости



числа из соответствующих статей; стоимость = GFLOPs/view × views

Смотрите на верхне-левую границу. X3D и MViT дешёвы при хорошей точности; VideoMAE наверху, но за счёт размера модели и предобучения.

7. Сводная карта подходов

Подход	Механизм по времени	Стоимость	Когда выбирать
2D CNN + avg	усреднение признаков кадров	дёшево	сильный статичный контекст, бейзлайн
Two-stream	RGB + optical flow	две ветви, дорогой flow	flow уже есть, нужна устойчивость к виду
C3D / I3D	локальная 3D-свёртка	$\approx k_t \times$ 2D-слоя	короткие локальные движения
R(2+1)D	spatial + temporal conv	та же ёмкость, легче обучать	практичная 3D CNN
SlowFast	медленная и быстрая ветви	вычисления неравномерны	баланс движения и семантики
TSN	K сегментов + консенсус	как 2D CNN на K кадрах	длинные ролики, дешёвый охват
TimeSformer / ViViT	факторизованное attention	растёт с числом токенов	дальние связи, много данных
Video Swin / MViT	окна, пирамида, pooling	контролируемая	универсальный трансформер
VideoMAE	восстановление по маске	дешёвый энкодер при 90% маске	много неразмеченного видео
V-JEPA	предсказание признаков	foundation-масштаб	перенос, семантика, планирование

7. Как выбрать модель на практике

Быстрый бейзлайн

2D ResNet / ConvNeXt по кадрам + average или TSN. Покажет, сколько решается одним внешним видом.

Дешёвое время

Лёгкая X3D, когда ограничены latency и GPU.

Движение важно

R(2+1)D, SlowFast или fine-tuning VideoMAE — по данным, compute и задержке.

Много неразмеченного видео

Предобучить VideoMAE или взять готовый SSL-энкодер: обычно лучше, чем учить трансформер с нуля на малом наборе.

Dense-задачи

Сегментация, глубина, трекинг: иерархические модели (Swin, MViT) и representation learning шире clip-level классификации.

Главные выводы

1. Видео-модель определяется тем, как она обрабатывает ось времени
2. 2D CNN + агрегация — сильный дешёвый бейзлайн; average/max движение почти не видят
3. 3D CNN учат пространственно-временные фильтры и платят множителем k_t на слой; I3D переносит ImageNet-веса инфляцией, R(2+1)D разделяет пространство и время
4. SlowFast, X3D: время и каналы можно обрабатывать неравномерно
5. Full attention стоит $O((THW)^2)$ — отсюда пространственно-временные патчи, факторизация, окна, пирамиды, pooling
6. VideoMAE: энкодер видит $\sim 10\%$ токенов; V-JEPA предсказывает признаки, а не пиксели
7. Высокий top-1 на Kinetics $\neq$ понимание движения: проверяйте Δ_{temporal} и SSv2 (Something-Something v2)
8. Честное сравнение фиксирует сэмплирование, кропы, число views, данные предобучения и latency; решать по Pareto-фронту

Литература

- ▶ **Simonyan, Zisserman** — Two-Stream ConvNets for Action Recognition, 2014
- ▶ **Tran et al.** — C3D, 2015; A Closer Look at Spatiotemporal Convolutions (R(2+1)D), CVPR 2018
- ▶ **Carreira, Zisserman** — Quo Vadis, Action Recognition? (I3D, Kinetics), CVPR 2017
- ▶ **Feichtenhofer et al.** — SlowFast, ICCV 2019; **Feichtenhofer** — X3D, CVPR 2020
- ▶ **Wang et al.** — TSN, ECCV 2016
- ▶ **Goyal et al.** — The «Something Something» Video Database (SSv2), ICCV 2017
- ▶ **Arnab et al.** — ViViT, ICCV 2021; **Bertasius, Wang, Torresani** — Is Space-Time Attention All You Need? (TimeSformer), ICML 2021
- ▶ **Fan et al.** — MViT, ICCV 2021; **Liu et al.** — Video Swin Transformer, CVPR 2022
- ▶ **Tong et al.** — VideoMAE, NeurIPS 2022; **Wang et al.** — VideoMAE V2, CVPR 2023; **Wang et al.** — InternVideo2, 2024
- ▶ **Bardes et al.** — V-JEPA, 2024; **Assran et al.** — V-JEPA 2, 2025; V-JEPA 2.1, 2026